

Design Pattern - RAI

Adam HAMOUC

May 2026

1. Intention

Le RAI (Resource Acquisition Is Initialization) est davantage un principe fondamental en C++ qu'un design pattern. Il consiste à gérer les ressources en fonction du cycle de vie des objets : on initialise les ressources dans le constructeur, et on les libère automatiquement dans le destructeur.

2. Structure

Le principe est simple :

- Allocation / acquisition de la ressource dans le constructeur
- Libération automatique dans le destructeur

Cela garantit que chaque ressource est correctement nettoyée, même en cas d'erreur ou d'exception.

3. Cas d'usage

À utiliser dès que l'on manipule des ressources nécessitant une gestion manuelle :

- Mémoire (new/delete)
- Fichiers
- Mutex / locks
- Ressources GPU (buffers, textures, etc.)

4. Avantages

Simplifie considérablement la gestion mémoire et des ressources en C++ :

- Évite les fuites mémoire
- Sécurise le code (exception-safe)
- Automatisation du cleanup
- Code plus lisible et maintenable

5. Inconvénients

- Demande une bonne compréhension du cycle de vie des objets
- Peut être mal utilisé avec des copies (shallow copy vs deep copy)
- Nécessite parfois des wrappers (smart pointers, classes dédiées)

6. Exemple minimal

```
class ObjetCPP {  
public:  
    ObjetCPP() {  
        ressource = createObjet();  
    }  
  
    ~ObjetCPP() {  
        freeObjet(ressource);  
    }  
  
private:  
    Objet* ressource;  
};
```

7. Exemple concret (jeu / moteur)

Exemples en moteur de jeu :

- Gestion automatique des textures (allocation GPU / libération)
- Buffers Vulkan / OpenGL
- Gestion de fichiers (chargement de ressources)
- Mutex pour le multithreading (std::lock_guard)

Le RAII est essentiel en engine programming pour éviter les fuites critiques de ressources (CPU/GPU).

8. Sources

<https://nodatek.com/articles/tuto-cpp-le-raii.htm>